

SE250B4

SDK User Guide

1VV0301823 Rev.4
2025-12-03
Released
NDA Restricted



Contents

1	Applicability Table	4
2	Introduction.....	5
2.1	Scope	5
2.2	Audience	5
2.3	Contact Information, Support	5
2.4	Conventions	5
2.5	Terms and Conditions	6
2.6	Disclaimer	6
3	General Product Description.....	7
3.1	Overview	7
3.2	High-Level System Architecture	7
3.3	Main Features	7
3.4	High-Level Software Architecture	9
4	Building Software.....	11
4.1	Host PC Requirements	11
4.2	Downloading Software and Patches	12
4.3	Setting Up Build Tools	12
4.4	Tools Required for Non-HLOS Software Components	14
4.5	Preparing Source Code	15
4.6	Building the Software	15
4.6.1	Steps to Build the Software	15
4.6.2	Building Individual Software Components	16
4.7	HLOS Firmware	16
4.8	Non-HLOS Firmware	17
4.9	Meta Build Procedure	17
5	Flashing and Testing Software.....	19
5.1	Installing ADB Drivers	19
5.2	Installing Microsoft Windows Flash tools	19
5.3	Flashing Binaries using QFIL	19
5.3.1	Prerequisites	19
5.3.2	Flashing Procedure	19
5.4	Flashing Binaries Using Fastboot	20
5.4.1	Prerequisites	21
5.4.2	Flashing Procedure	21
5.5	Bootting and Testing Software	21
6	Building FOTA (Firmware Over the Air) Images.....	22
6.1	Generating OTA Package	22
6.1.1	Creating a full OTA Upgrade Package	22
6.1.2	Creating an Incremental OTA Upgrade Package	23
6.2	Testing OTA Upgrade	23
6.3	Adding an HLOS Image to the OTA Upgrade Package	23
6.4	Adding a non-HLOS Image to OTA Upgrade Package	24
6.4.1	Building ADSP Binary	24
6.4.2	Building devcfg Binary	24
6.4.3	Building XBL Binary	24
6.4.4	Generating Updated Target Files	25

6.5	Build Script Changes to Include an Image as Part of the OTA	25
7	Application Development.....	26
8	Android Customization	27
8.1	Case 1: Removing an Application by Modifying base.mk File	27
8.2	Case 2: Adding an APK with Source Code into the System	27
8.3	Case 3: Adding an APK Without Source Code into the System	29
9	Acronyms and Abbreviations.....	31
10	Related Documents	32
11	Document History.....	33

NDA Restricted

1 Applicability Table

Table 1: Applicability Table

Products
SE250B4-NA
SE250B4-EU

NDA Restricted

2 Introduction

2.1 Scope

This document introduces the Telit SE250B4 Smart module and presents possible and recommended hardware solutions for developing a product based on this module. All the features and solutions detailed in this document apply to all Telit SE250B4 Smart module variants.

This document cannot embrace every hardware solution or every product that can be designed. Where the suggested hardware configurations need not be considered mandatory, the information given should be used as a guide and a starting point for properly developing your product with the Telit module.

2.2 Audience

This document is intended for system integrators that are using the Telit SE250B4 module in their products.

2.3 Contact Information, Support

For technical support and general questions, e-mail:

- TS-EMEA@telit.com
- TS-AMERICAS@telit.com
- TS-APAC@telit.com
- TS-SRD@telit.com
- TS-ONEEDGE@telit.com

Alternatively, use: <https://www.telit.com/contact-us/>

For Product information and technical documents, visit: <https://www.telit.com>

2.4 Conventions

Note: Provide advice and suggestions that may be useful when integrating the module.

Danger: This information **MUST** be followed, or catastrophic equipment failure or personal injury may occur.

Warning: Alerts the user on important steps about the module integration.

All dates are in ISO 8601 format, that is YYYY-MM-DD.

2.5 Terms and Conditions

Refer to <https://www.telit.com/terms-conditions/>.

2.6 Disclaimer

THE MATERIAL IN THIS DOCUMENT IS FOR INFORMATIONAL PURPOSES ONLY. TELIT CINTERION RESERVES THE RIGHT TO MAKE CHANGES TO THE PRODUCTS DESCRIBED HEREIN. THE SPECIFICATIONS IN THIS DOCUMENT ARE SUBJECT TO CHANGE AT THE DISCRETION OF TELIT CINTERION WITHOUT PRIOR NOTICE. THIS DOCUMENT IS PROVIDED ON "AS IS" BASIS ONLY AND MAY CONTAIN DEFICIENCIES OR INADEQUACIES. TELIT CINTERION DOES NOT ASSUME ANY LIABILITY FOR INFORMATION PROVIDED IN THE DOCUMENT OR ARISING OUT OF THE APPLICATION OR USE OF ANY PRODUCT DESCRIBED HEREIN.

TELIT CINTERION GRANTS A NON-EXCLUSIVE RIGHT TO USE THE DOCUMENT. THE RECIPIENT SHALL NOT COPY, MODIFY, DISCLOSE, OR REPRODUCE THE DOCUMENT EXCEPT AS SPECIFICALLY AUTHORIZED BY TELIT CINTERION.

TELIT CINTERION AND THE TELIT CINTERION LOGO, ARE TRADEMARKS OF TELIT CINTERION AND ARE REGISTERED IN CERTAIN COUNTRIES. ALL OTHER REGISTERED TRADEMARKS OR TRADEMARKS MENTIONED IN THIS DOCUMENT ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS AND ARE EXPRESSLY RESERVED BY TELIT CINTERION (AND ITS LICENSORS).

3 General Product Description

3.1 Overview

The Telit SE250B4 Smart module is the latest addition to Telit's IoT portfolio. It is a multi-mode and multi-band wireless smart module built on the Qualcomm QCM2290 platform. The QCM2290 chip is the next-generation IoT platform based on the Qualcomm Snapdragon processor and LTE modem. It is designed with an 11 LPP process for lower active power dissipation and faster CPU performance.

The module integrates baseband, memory, wireless connectivity, and an RF front end while supporting multiple air interface standards, including GSM, WCDMA, and LTE networks.

With its compact design, the Telit SE250B4 Smart module enables seamless integration, reduces PCB footprint and BOM costs, accelerates time-to-market, and simplifies adoption across various industries.

3.2 High-Level System Architecture

3.3 Main Features

Table 2: Main Features

Function	Features
Application processor	Quad ARM Cortex-A53 cores up to 2.0GHz, 64-bit Arm Cortex
Memory RAM	2GB, 1804MHz clock (2x16 bits), LPDDR4X SDRAM
Memory Flash	16GB (default) SE250B4-NA / SE250B4-EU External memory via SDC2 Support SD3.0 flash devices, up to 128GB
Graphics	Adreno 702; up to 845MHz with 64-bit addressing
Operating system	Android 13 or above
Software upgrade	Upgrade via USB interface; support forced download
Power supply	Voltage range: 3.4V ~ 4.4V, support single-cell lithium battery power supply
Charge management	Integrated up to 1.85A for fast charging
Display support	4-lane MIPI_DSI, 1.5Gbps each; HD+(1680x720), 60fps
Camera interfaces	MIPI_CSI – up to 3 cameras (4/2/1), 25MP total max
Video applications performance	Encode: 1080p 30 fps: H265 Decode: 1080p 30 fps: HEVC, H.265, VP9 Encode + Decode: 720p 30 fps + 1080p 30 fps
Audio supply	Three analog inputs that support single-ended configurations OR two data input Three outputs: earpiece, stereo headphones, and mono speaker driver
Audio codec support	MP3; AAC; He-AAC v1, v2; WMA 9/Pro; Dolby AC-3, eAC-3, DTS-HD M6 and DTS-HD M8
Voice codec support	EVRC, -B, -WB; G.711; G.729A,-AB; GSM-FR,-EFR,-HR; ARM-NB,-WB; eAMR; BeAMR
USB	USB 3.1/2.0 compliant
UART	Supports up to 3*UART, up to 4MHz
I2C	Supports up to 6*I2C, used to cameras, sensors, TP, and so forth

Function	Features
SPI	Supports up to 5*SPI, up to 50MHz (Master only)
ADC	Supports up to 2*ADC, typical input range: 0.3~4.5V
UIM card	Dual cards dual standby; 1.8V/2.95V dual voltage adaptation
LTE features	Support 3GPP R12 CAT4 FDD and TDD Support 1.4 to 20MHz RF bandwidth Cat 4 FDD Datarate Max 150Mbps (DL) / Max 50Mbps (UL) Cat 4 TDD Datarate Max 130Mbps (DL) / Max 30Mbps (UL)
UMTS features	Support 3GPP R9 DC-HSDPA/HSPA+/HSDPA/HSUPA/WCDMA Support 16-QAM, QPSK DC-HSDPA: Max 42Mbps (DL) HSUPA: Max 5.76Mbps (UL) WCDMA: Max 384Kbps (DL) / Max 384Kbps (UL)
GSM features	R99: CSD: 9.6kbps, 14.4kbps GPRS: Support GPRS multi-slot level 33 (default 33) coding schemes: CS-1, CS-2, CS-3, CS-4 Max 85.6Kbps (UL) / Max 107Kbps (DL) EDGE: Support EDGE multi-slot level 33 (default 33) Support GMSK and 8-PSK Modulation coding DL coding schemes: CS 1-4 and MCS 1-9 UP coding schemes: CS 1-4 and MCS 1-9 Max 236.8Kbps (UL) / Max 296Kbps (DL)
WLAN features	Support SoftAP Function, 802.11 a/b/g/n/ac Encryption: WFA WPA/WPA2 Qos: WFA WMM, WMM PS Wi-Fi bands: 2.4GHz/5GHz
BT Features	Bluetooth 5.0 and earlier Specification: V2.1+EDR, 3.0+HS, V4.1 BLE Tx power levels: Class 1 & 2

The system architecture of the Telit SE250B4 Smart module includes the following components:

- Application processor
- Memory
- Camera
- Display
- USB 3.1
- Peripherals: UART, SPI, eMMC, μ SD (micro-SD card), USB
- Connectivity: Wi-Fi 802.11 (a/b/g/n/ac) and Bluetooth 5.0
- Audio output
- RPM: Cortex M3
- Modem: LTE Cat 4

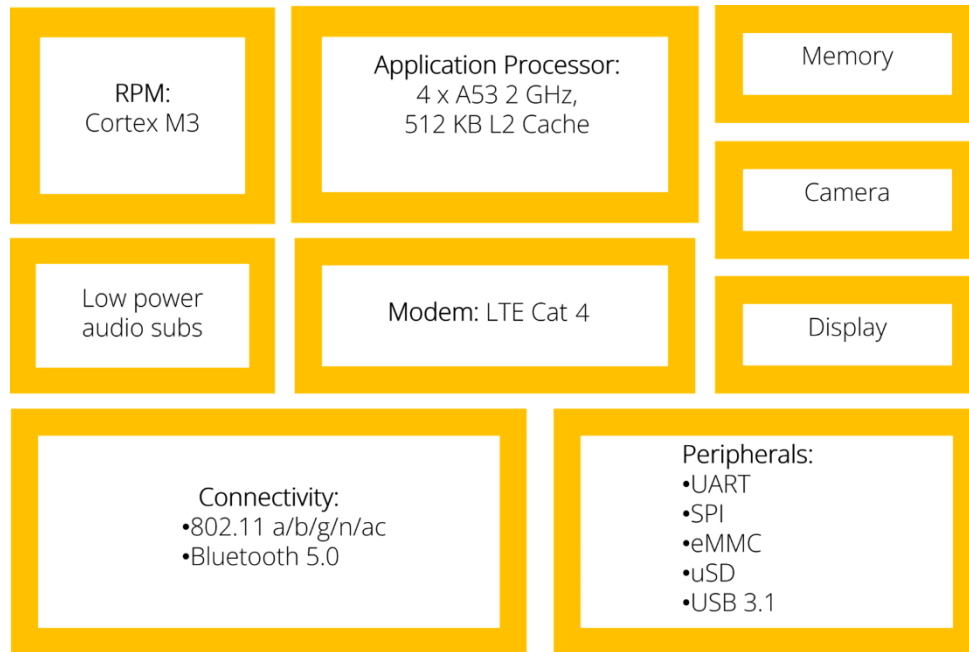


Figure 1: Telit SE250B4 Smart Module High-Level System Architecture

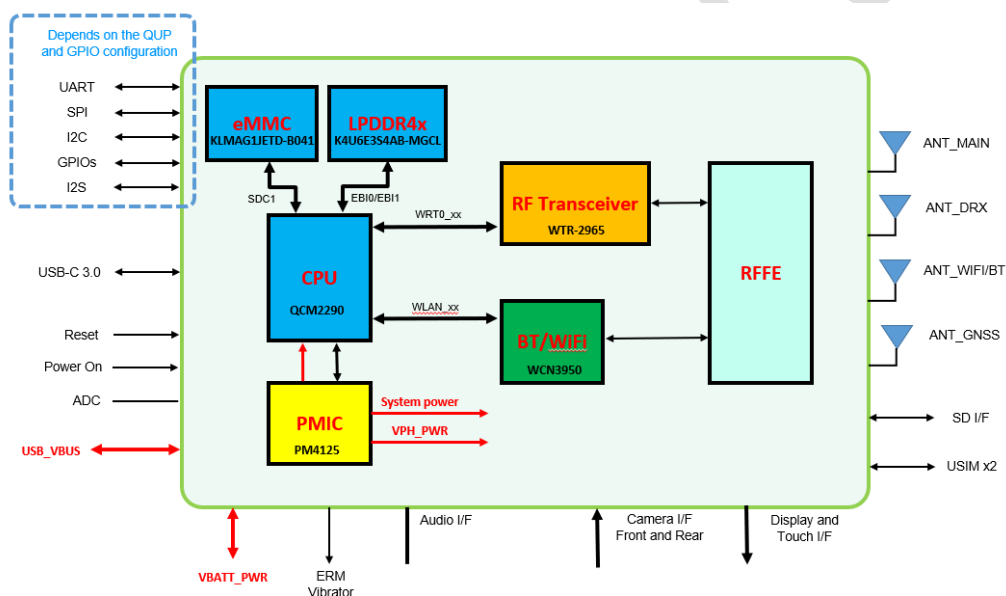


Figure 2: Telit SE250B4 Smart Module Block Diagram

3.4 High-Level Software Architecture

The SE250B4 module operates on the Android operating system, leveraging an application processor with 4xA53 cores @ 2 GHz. It is an ideal platform for developing high-performance, low-power, and cost-effective IoT devices. The software on the SE250 offers the following key features:

- A development and debug environment based on a fully integrated and tested Android framework.
- Android framework based on a standard and stable Android release.
- Patches for Telit proprietary updates and customizations.
- Reference Sensor applications – Light sensor, Proximity sensor.
- FOTA for updating firmware in the field.
- LwM2M protocol modules.

- Programming tools to program firmware during development.

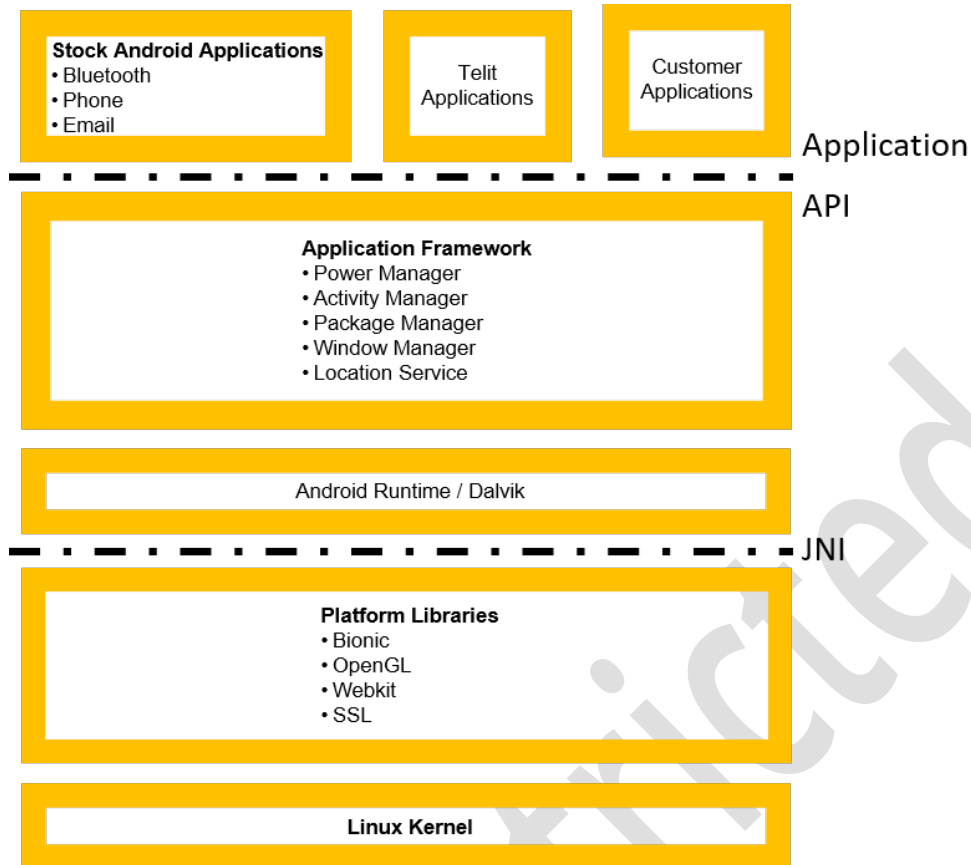


Figure 3: Telit SE250B4 Smart Module High-Level Software Architecture

For more information related to Android architecture, refer to <https://source.android.com/devices/architecture>

Table 3: Development Tools

Component	Description
Application development	To develop applications on top of an application framework. For more information, see Application Development .
Driver Development	For more information, see Application Development .
Android Customization	To develop customised Android solutions. For more information, see Android Customization .

4 Building Software

The SDK package includes source code for the following components:

- 1 HLOS (High-Level Operating System): This is the Android-based software. It includes the Linux kernel, the full AOSP source code including Android framework and HAL, the vendor-specific HAL software, and the user applications built as part of the firmware build. Customers can modify any part of the HLOS software.
- 2 Non-HLOS (Non-High Level Operating System): The source code for the following Non-HLOS components is also provided as part of the SDK.
 - XBL (eXtended Boot Loader): This is available in the BOOT.XF.4.1 package. The XBL source code is provided for the following reasons:
 - To customise the splash image
 - For including Android verified boot certificates needed for secure boot
 - TZ (TrustZone): Available in the TZ.XF.5.1 package. The TZ source code is provided for the following reasons:
 - To modify IO QUP engine assignment
 - ADSP (Audio subsystem DSP): This is available in the ADSP. VT. 5.4.1 package. The ADSP source code is provided for the following reasons:
 - To add support for new sensors

Note: The non-HLOS source code does not need to be compiled as part of sdk build if none of the above customizations are required. The pre-Build Non-HLOS binaries are available in the prebuilts folder within the Telit release package. For example, in SE250B5_42.xx.xx/SE250B4/prebuilts and can be used as is. The prebuilt non-HLOS binaries are provided for both EU and NA variants, and are available for both release and userdebug builds. choose the appropriate set based on the variant and build type. This is explained further in the build procedure.

4.1 Host PC Requirements

The following is an example of an Android compilation environment:

Table 4: Host PC Requirements

Component	Specification
CPU	Intel® Quad Core™ i7-2600 @ 3.4 GHz (or better)
Memory	16GB RAM minimum
Hard Disk	1TB
Ubuntu	Ubuntu 20.04 LTS (64-bit)

4.2 Downloading Software and Patches

- The **SE250B4_ENV.tar.gz** package provides the portable build environment required to build the SE250B4 SDK source code.
- The **SE250B4_42.xx.xxx.tar.gz** Release package contains patch files for both HLOS and Non-HLOS subsystems' source codes.

4.3 Setting Up Build Tools

Configure the build tools using the **portable environment**, which allows the creation of different Ubuntu versions and build environments on a single machine.

To set up the portable build environment, follow these steps:

- 1 Run the following commands to install the necessary packages:

```
$ sudo apt-get install schroot
```

Note: In the new system, there is no **.bashrc** file, and user environment variables are not shared, nor automatically loaded. Source the environment variable file manually (Copy the **.bashrc** file from the original system to the new system.)

- 2 Extract the environment package into the **/srv/chroot** directory.

```
$ sudo mkdir -p /srv/chroot
```

```
$ sudo tar -zxvf ~/SE250B4_ENV.tar.gz --directory=/srv/chroot/
```

```
$ mv /srv/chroot/SE250B4_ENV/se250b4/ /srv/chroot/
```

Note: If the **/srv/chroot** folder is not present in your system, then manually create the **chroot** folder. After extraction, the portable build environment will be available at **/srv/chroot/se250b4/**

- 3 Modify the schroot configuration file (**/etc/schroot/schroot.conf**) by adding the following entry:

```
[se250b4]
Description=se250b4 build environment
Type=directory
directory=/srv/chroot/se250b4
users=<user_name>
groups=<user_name>
```

```
# schroot chroot definitions.
# See schroot.conf(5) for complete documentation of the file format.
#
# Please take note that you should not add untrusted users to
# root-groups, because they will essentially have full root access
# to your system. They will only have root access inside the chroot,
# but that's enough to cause malicious damage.
#
# The following lines are examples only. Uncomment and alter them to
# customise schroot for your needs, or create a new entry from scratch.
#
#
[se250b4]
description=se250b4 build environment
type=directory
directory=/srv/chroot/se250b4
users=test_user
groups=test_user
~
~
~
~
~
~
```

Figure 4: Sample Configuration

Note: In the above sample configuration, we have used “test_user” as the user name of the PC. Use an appropriate user name as per your host PC.)

4 Modify the schroot fstab file (/etc/schroot/default/fstab).

Uncomment the following entries to allow runtime access:

```
/run,
/run/shm,
/dev/shm
/run/lock
```

```
1 # fstab: static file system information for chroots.
2 # Note that the mount point will be prefixed by the chroot path
3 # (CHROOT_PATH)
4 #
5 # <file system> <mount point> <type> <options> <dump> <pass>
6 /proc /proc none rw,bind 0 0
7 /sys /sys none rw,bind 0 0
8 /dev /dev none rw,bind 0 0
9 /dev/pts /dev/pts none rw,bind 0 0
10 /home /home none rw,bind 0 0
11 /tmp /tmp none rw,bind 0 0
12 # It may be desirable to have access to /run, especially if you wish
13 # to run additional services in the chroot. However, note that this
14 # may potentially cause undesirable behaviour on upgrades, such as
15 # killing services on the host.
16 #/run /run none rw,bind 0 0
17 #/run/lock /run/lock none rw,bind 0 0
18 #/dev/shm /dev/shm none rw,bind 0 0
19 #/run/shm /run/shm none rw,bind 0 0
20 /srv/chroot/se250b4 /srv/chroot/se250b4 none rw,bind 0 0
21 /run /run none rw,bind 0 0
22 /run/lock /run/lock none rw,bind 0 0
23 /dev/shm /dev/shm none rw,bind 0 0
24 /run/shm /run/shm none rw,bind 0 0
```

Figure 5: Fstab Entry

- 5 Modify `/etc/schroot/default/copyfiles`, to copy the `resolv.conf` and `localtime` setting into the chroot environment.

```
# Files to copy into the chroot from the host system.
#
# <source-and-destination>[ <destination>]
/etc/resolv.conf
/etc/localtime
~
~
~
```

Figure 6: Copyfiles Entry

- 6 Usage Instructions:

- To access the system, run:

```
$ schroot -c se250b4 -u <user_name>
```

Example: If your username of the system is “test_user”, use:

```
$ schroot -c se250b4 -u test_user
```

- To configure username and email for Git:

```
git config --global user.email "your@example.com"
```

```
git config --global user.name "Your Name"
```

- 7 To exit the environment package, run:

```
Exit
```

4.4 Tools Required for Non-HLOS Software Components

The SE250B4 portable build environment also includes the following tools required to build the following non-HLOS subsystem source codes:

- XBL (eXtended Boot Loader): Qualcomm Snapdragon LLVM Arm compiler 4.0.2 is required to compile BOOT.XF.4.1 source code.
- TZ (TrustZone): Snapdragon LLVM Arm compiler 8.0.9 and GNU Linaro Cross-compile tool chain v4.9 is required to build Trustzone TZ.XF.5.1 source code.
- ADSP (Audio subsystem DSP): Qualcomm Hexagon DSP Toolchain 8.2.05 is required to build ADSP.VT.5.4.1 source code.

Note: The non-HLOS source code does not need to be built as part of sdk build if they are not modified.

The pre-built non-HLOS binaries are available in the prebuilts folder within the release package. for example, in `se250b4_42.xx.xx/se250b4/prebuilts` and can be used as is.

the prebuilt non-HLOS binaries are provided for both EU and NA variants and are available for both release and userdebug builds. choose the appropriate set based on the variant and build type. this is explained further in the build procedure.

4.5 Preparing Source Code

The SE250B4 source code consists of the following components:

- **Qualcomm QCS2290 Source Code:**
This serves as the base source code, obtained from Qualcomm chipcode corresponding to the Telit release in use.
- **AOSP Source Code:**
This is updated on top of the QCS2290 base code using repo commands. The next section describes the steps and commands to update this source code.
- **Telit Custom Patches and Source Code:**
Telit provides this and must be applied on top of the QCS2290 base code and AOSP source code.

Steps to Prepare the Source Code

Refer to the “Steps to Prepare the Source Code” section in the release notes.

After preparing the source code as per the release notes, continue as follows.

4.6 Building the Software

The SE250B4 source code provides the scripts to build HLOS and other non-HLOS subsystems such as BOOT, ADSP, and TZ.

Note: Switch to root using the “su” command for administrative privilege

4.6.1 Steps to Build the Software

- 1 Enter the portable build environment:

```
$ schroot -c se250b4 -u <user_name>
$ cd ~/se250b4_build/se250b4/build_scripts
```

- 2 Execute the build command:

```
$ ./build_system_ext.sh --model=[0UE0|0UWW] --build=[ALL | ADSP
BOOT | TZ | HLOS]
```

Note: If the --build option is not specified, the HLOS build runs by default.

Examples:

- For the NA module userdebug build:

```
$ ./build_system_ext.sh --model=0UWW --build=ALL
```

- For EU module userdebug build:

```
$ ./build_system_ext.sh --model=0UE0 --build=ALL
```

- For NA module release build:

```
$ ./build_system_ext.sh --model=00WW --build=ALL
```

- For EU module release build:

```
$ ./build_system_ext.sh --model=00E0 --build=ALL
```


- Note:** After compilation, the generated image files will be located at:
- HLOS Images:
'LA.UM.9.15.2/LINUX/android/out/target/product/bengal_32go'
 - ADSP, BOOT, TZ build images: Respective "out/" directories.

4.6.2 Building Individual Software Components

Individual software packages can be built by selecting the appropriate model and build options.

- Build only the Boot Loader (EU module):

```
$/build_system_ext.sh --model=00E0 --build=BOOT
```

- Build only ADSP (EU module):

```
$/build_system_ext.sh --model=00E0 --build=ADSP
```

- Build only TrustZone (EU module)

```
$/build_system_ext.sh --model=00E0 --build=TZ
```

- Build only HLOS (EU module):

```
$/build_system_ext.sh --model=00E0 --build=HLOS
```

Similarly, to build NA software images, use model=00WW.

4.7 HLOS Firmware

The compiled binaries are generated in the following path:

```
LA.UM.9.15.2/LINUX/android/out/target/product/bengal_32go/
```

The following figure illustrates the list of files generated after the successful compilation of the Android source code.

The HLOS binaries can be programmed using fastboot, following the procedure in the section [5.4 Flashing Binaries Using Fastboot](#).

Refer to the following.

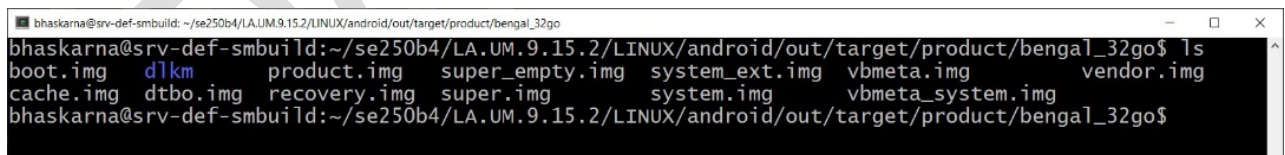


Figure 7: HLOS firmware

Refer to the following table for Image file names and their descriptions.

Table 5: Image File Name Descriptions

File Name	Description
boot.img	Including core start parameter, core, and multiple elements.
cache.img	Contains /cache file system
dtbo.img	Device tree blob (DTB) image
product.img	This image contains product-specific features
recovery.img	The image package needed for the device entering 'recovery mode'
super.img	It contains system.img, vendor.img etc, in addition to the super partition metadata.

File Name	Description
system.img	Android system executing package (framework is here), it will be loaded to the device /system directory
system_ext.img	Contains extension and custom features that have tight coupling with the components in the /system partition
vbmeta.img	Its signed image contains verification data for verifying boot.img and other partitions/images.
vbmeta_system.img	This image contains the hash-tree descriptors for the system.img
vendor.img	Vendor image contains device-specific hardware drivers

4.8 Non-HLOS Firmware

The compiled Non-HLOS binaries are generated in their respective source code “out” folders.

If none of the Non-HLOS source is modified and compiled, then the Non-HLOS binaries available in **prebuilts** folder within the Telit release package **MUST** be used. The prebuilts folder “**SE250B4_42.xx.xx/se250b4/prebuilts**” can be found in the release package.

The prebuilt Non-HLOS binaries are provided for both EU and NA regions, and are available for both release and userdebug builds. Choose the appropriate set based on the variant and build type.

Table 6: Non-HLOS Related Files and Descriptions

File Name	Description
dspso.bin	DSP bottom layer file.
gpt_backup0.bin	Backup GPT head and partition table.
gpt_main0.bin	Main GPT head and partition table.
NON-HLOS.bin	The modem image is responsible for managing the baseband image of the communication protocol.
patch0.xml	It is related to the partition table, mainly about the GPTT partition table head and partition item contents.
prog_firehose_ddr.elf	Used when using QFIL software to burn the system, it is related to EMMC and DDR configuration parameters.
rawprogram_unsparse0.xml	Generated by partition.xml, which is used to choose the partition table during flashing, and is used together with patch0.xml.
rpm.mbn	Resource Power Manager is a chipset added to the Qualcomm MSM platform, although it is packaged with an AP chipset, it is an independent ARM Core. It controls the related shared resources of the entire power supply, for example, Ido, clock. It is in charge of interacting with SMP, MPM to enter sleeping mode or wake up the whole system
tz.mbn	ARM TrustZone® technology is the system scope safe method, based on safety requirements and leading mode configuration XPU, NAND MPU.

4.9 Meta Build Procedure

The meta build procedure is an additional step required to prepare all the necessary binaries for programming using the QFIL tool. This is needed while building for the first time or if any Non-HLOS source is modified and compiled. If any Non-HLOS source is modified and compiled, then use the Meta build procedure and program using QFIL

This step is not necessary if only the HLOS code is modified, built and programmed using fastboot. Therefore, the HLOS code can be modified, built and programmed over and over again using fastboot without doing a meta build.

To generate meta build firmware images, follow these steps:

Navigate to the build scripts directory and execute the meta build script:

```
cd ~/se250b4_build/se250b4/build_scripts
./meta_build.sh
```

After execution, the generated HLOS and Non-HLOS binaries will be available at: `~/se250b4_build/se250b4/built_binaries` folder.

Additionally, **built_binaries.zip** will be located at `~/se250b4_build/se250b4/built_binaries.zip`.

The binaries generated from the meta build procedure and the non-HLOS binaries from the “prebuilts” folder of the Telit release package together form the complete set of binaries required for SE250B4.

To get all the binaries for programming the device using QFIL tool on Windows PC, do the following.

First copy all the binaries from the **prebuilts** folder to the folder where you are collecting all the binaries. Then copy all the binaries from `~/se250b4_build/se250b4/built_binaries`, overwriting some of the files copied earlier from the **prebuilts** folder.

The final folder containing all the binaries would resemble the structure below.

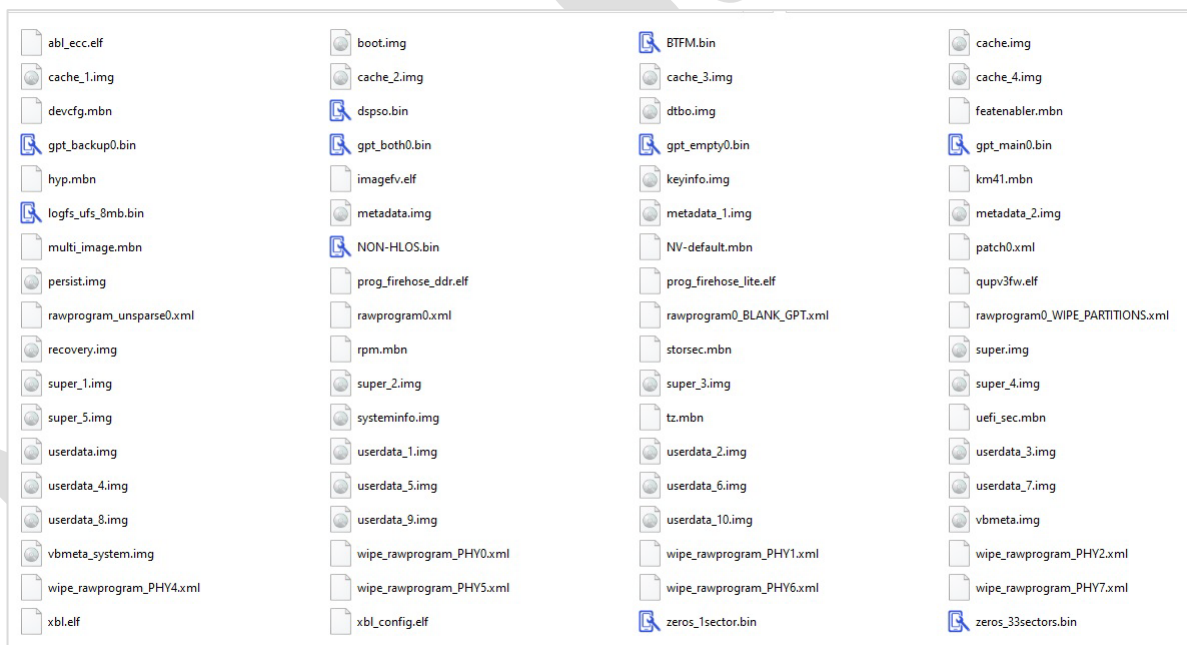


Figure 8: Binaries in Windows PC

5 Flashing and Testing Software

5.1 Installing ADB Drivers

For information about Android SDK Platform-Tools, visit:

<https://developer.android.com/studio/releases/platform-tools>

5.2 Installing Microsoft Windows Flash tools

- 1 Install QPST version 2.7.496 or later.
- 2 Ensure QFIL version 2.0.3.5 is included in the installation.

5.3 Flashing Binaries using QFIL

Before flashing binaries using QFIL, the firmware must be built following the build procedure described earlier, including meta build procedure. To use QFIL Tool to burn binaries, follow the steps below:

5.3.1 Prerequisites

- Install QPST Software and Qualcomm drivers on the PC.
- Ensure the device is powered on.
- Connect the device to the PC using a USB cable.

5.3.2 Flashing Procedure

- 1 Launch the QFIL tool.
- 2 Click **Select Port** and choose **port 9008** for loading; (Use `adb reboot edl` in the terminal).
- 3 Under **Build Type**, select **Flat Build**.

Note: In case switching to EDL fails, put the device into forced boot mode by shorting the jumper at EDL on the TE board.

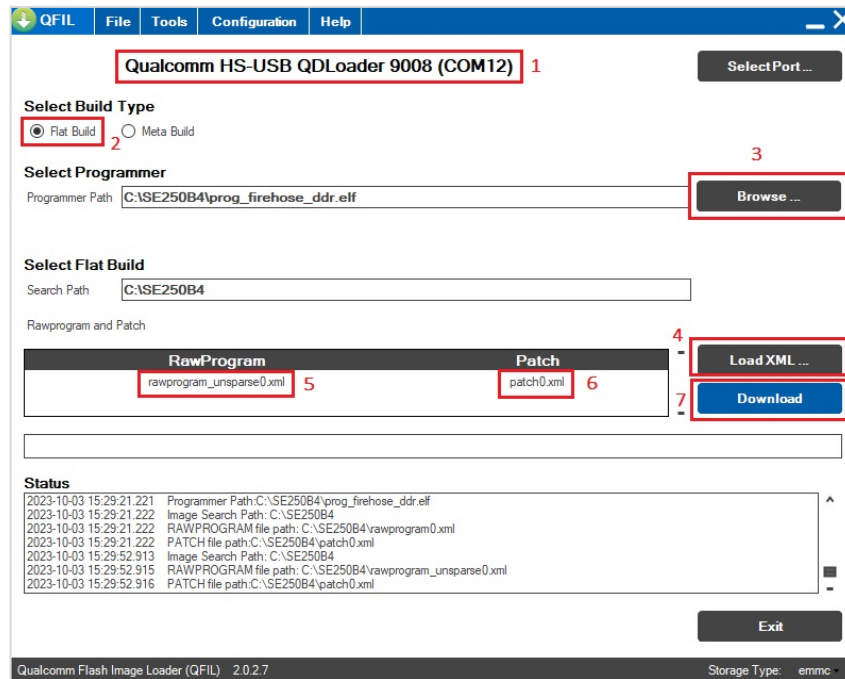


Figure 9: Install Steps

- 4 Click **Browse** and select the `prog_firehose_ddd.elf` file.
- 5 Click **Load XML** and select the `rawprogram_unsparse0.xml` and the `patch0.xml` file.
- 6 Click **Download**.

After successful completion, the following message is displayed.

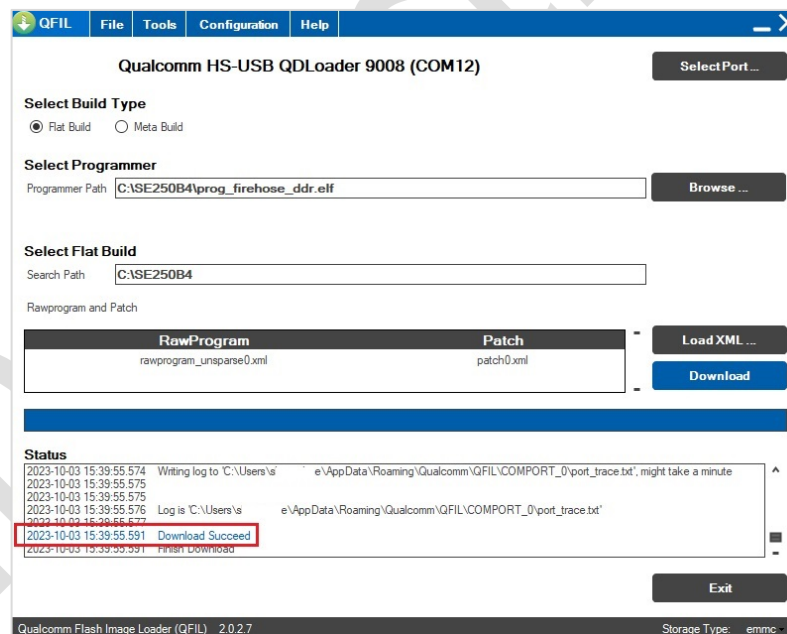


Figure 10: XML Success Message

- 7 Reset the device.

5.4 Flashing Binaries Using Fastboot

The fastboot flashing procedure can be used to flash HLOS binaries when only HLOS images are built using the procedure described in the section [4.7 HLOS Firmware](#).

5.4.1 Prerequisites

Before programming the system images using Fastboot, ensure the following:

- The device must be flashed at least once using the QFIL tool.
- Ensure the device is in fastboot mode.

5.4.2 Flashing Procedure

- 1 To enter fastboot mode, execute the following command:

```
adb reboot bootloader
```

- 2 Connect the device to the PC via USB cable.
- 3 Depending on your environment, choose one of the following options:
 - From Windows, run the following command:

```
fastboot devices
```

- From Linux, run the following command:

```
sudo fastboot devices
```

A list of registered devices is displayed.

- 4 Once the device is detected, flash the binaries to the target using the following commands:

Fastboot flash image commands:

```
fastboot flash boot "boot.img"
```

```
fastboot flash recovery "recovery.img"
```

```
fastboot flash vbmeta_system "vbmeta_system.img"
```

```
fastboot flash vbmeta "vbmeta.img"
```

```
fastboot flash dtbo "dtbo.img"
```

```
fastboot flash super "super.img"
```

```
fastboot -w
```

- 5 Restart the module.

```
fastboot reboot
```

5.5 Booting and Testing Software

- 1 Connect the power source.
- 2 Connect the USB cable.
- 3 Click **Settings**.
- 4 Click **Device**.
- 5 The updated build number is now visible.

6 Building FOTA (Firmware Over the Air) Images

This chapter explains the steps to be followed to generate and test OTA update packages for SE250B4.

- Android devices in the field can receive and install [over-the-air](#) (OTA) updates to the system, app software, and time zone rules.
- OTA updates are designed to upgrade the underlying operating system, the read-only apps installed on the system partition, and time zone rules; these updates do *not* affect apps installed by the user.
- There are two ways to upgrade the target: either with a full OTA package or with an incremental OTA package.
- A *full* update is an OTA package that contains the entire final state of the device (system, boot, and recovery partitions). If the device can receive and apply the package, the package can install the build regardless of the device's current state.
- An *incremental* update is an OTA package that contains binary patches to data already on the device. Packages with incremental updates are typically smaller as they don't need to include unchanged files. In addition, as changed files are often very similar to their previous versions, the package only needs to include an encoding of the differences between the two files.
- An incremental update package can be installed only on devices that have the source build used in constructing the package.

6.1 Generating OTA Package

- The Over the Air (OTA) package tool (`ota_from_target_files`) is provided in `LA.QSSI.13.0/LINUX/android/build/make/tools/releasetools` is used to generate the update packages.
- Firmware OTA (FOTA) images are generated using the target files.
- Whenever the source code is compiled with the below command, the target file also gets generated.
 - `./build_system_ext.sh -model=0UE0 -build=ALL`
 - Select the model as per the build variant.
- The merged-qssi_32go_bengal_32go-target_files.zip file will be generated at `LA.UM.9.15.2/LINUX/android/out/dist/` path.

6.1.1 Creating a full OTA Upgrade Package

- Run the following commands to generate the update.zip file,
 - `cd LA.QSSI.13.0/LINUX/android/`
 - `source build/envsetup.sh`
 - `lunch qssi_32go-userdebug`
 - `ota_from_target_files path_to_merged-qssi_32go_bengal_32go-target_files.zip path_to_update.zip`

6.1.2 Creating an Incremental OTA Upgrade Package

- Generate the merged-qssi_32go_bengal_32go-target_files.zip files from the two builds.
 - merged-qssi_32go_bengal_32go-target_files_v1.zip - Represents the old build on the device.
 - merged-qssi_32go_bengal_32go-target_files_v2.zip - Represents the new build to be pushed into the device.
- Run the following commands to generate the update.zip file,
 - `cd LA.QSSI.13.0/LINUX/android/`
 - `source build/envsetup.sh`
 - `lunch qssi_32go-userdebug`
 - `ota_from_target_files -i path_to_merged-qssi_32go_bengal_32go-target_files_v1.zip path_to_merged-qssi_32go_bengal_32go-target_files_v2.zip path_to_update.zip`

6.2 Testing OTA Upgrade

- OTA update can be tested using adb tool.
- Push the update.zip file to the target using adb.
 - `adb root`
 - `adb push path_to_update.zip /data/fota/`
- Initiate the OTA upgrade
 - `adb shell am broadcast -a com.android.intent.otauupdate --es path /data/fota/update.zip`
- If the update.zip is valid, target device boots with the latest update.
- For final product deployment, use the FOTA App or any other android implementation.
- For reference, Telit FOTA App is installed in a device that can be used to test OTA.

6.3 Adding an HLOS Image to the OTA Upgrade Package

- Below is the list of HLOS images that are patched as part of update.zip,
 - system.img
 - vendor.img
 - product.img
 - system_ext.img
 - boot.img
 - vbmeta.img
 - vbmeta_system.img
 - recovery.img
- Any HLOS image excluding the above list, which is supposed be updated as part of the OTA upgrade must be included in the update.zip file.
- Make sure the HLOS image which is supposed to be updated as part of OTA has an entry in the filesmap file located at,
 - `LA.UM.9.15.2/LINUX/android/device/qcom/bengal_32go/radio/`
 - Example: To include recovery.img, filesmap should have below partition information entry,

- recovery.img /dev/block/bootdevice/by-name/recovery
- Unzip the merged-qssi_32go_bengal_32go-target_files.zip file into a temporary directory.
 - cd LA.UM.9.15.2/LINUX/android/out/dist/
 - unzip -q merged-qssi_32go_bengal_32go-target_files.zip -d temp/
- Copy the HLOS image to be included in OTA upgrade package into the RADIO path of the temporary directory.
 - Example: To update the recovery.img,
 - cp recovery.img LA.UM.9.15.2/LINUX/android/out/dist/temp/RADIO/
- Generate the updated merged-qssi_32go_bengal_32go-target_files.zip with newly included images.
 - cd LA.UM.9.15.2/LINUX/android/out/dist/temp/
 - zip -qur LA.UM.9.15.2/LINUX/android/out/dist/merged-qssi_32go_bengal_32go-target_files.zip ./*
- Generate the full or incremental OTA upgrade package and push it into the target.

6.4 Adding a non-HLOS Image to OTA Upgrade Package

- Telit SDK supports modification and building of the following non-HLOS software components:
 - aDSP – analog DSP for adding support for new sensors
 - devcfg – for changing IO engines' assignments
 - XBL – eXtended Boot Loader for modifying the boot loader and customizing splash images.
- Below is the list of non-HLOS images which can be upgraded as part of OTA,
 - adsp.mbn
 - devcfg.mbn
 - xbl.elf

6.4.1 Building ADSP Binary

- adsp.mbn is part of NON-HLOS.bin and to update adsp.mbn, latest NON-HLOS.bin must be generated.
- Follow the commands below to generate adsp.mbn and NON-HLOS.bin
 - cd build_scripts/
 - ./build_system_ext.sh -model=0UE0 -build=ADSP
 - ./meta_build.sh

6.4.2 Building devcfg Binary

- devcfg.mbn is built as part of the Trustzone (TZ) source code.
- Follow the commands below to build TZ codebase and generate devcfg.mbn
 - cd build_scripts/
 - ./build_system_ext.sh -model=0UE0 -build=TZ

6.4.3 Building XBL Binary

- Follow commands below to generate xbl.elf.
 - cd build_scripts/
 - ./build_system_ext.sh -model=0UE0 -build=BOOT

6.4.4 Generating Updated Target Files

- Make sure the non-HLOS image, which is supposed to be updated as part of OTA has an entry in the *filesmap* file located at,
 - LA.UM.9.15.2/LINUX/android/device/qcom/bengal_32go/radio/
 - Below are the examples for the entries in filesmap. These changes are already part of Telit Patches.
 - To include NON-HLOS.bin, filesmap should have below partition information entry,
 - NON-HLOS.bin /dev/block/bootdevice/by-name/modem
 - To include *devcfg.mbn*, filesmap should have the following partition information entry,
 - devcfg.mbn /dev/block/bootdevice/by-name/devcfg
 - To include *xbl.elf*, filesmap should have below partition information entry,
 - xbl.elf /dev/block/bootdevice/by-name/xbl
- Unzip the merged-qssi_32go_bengal_32go-target_files.zip file into a temporary directory.
 - cd LA.UM.9.15.2/LINUX/android/out/dist/
 - unzip -q merged-qssi_32go_bengal_32go-target_files.zip -d temp/
- Copy the non-HLOS image to be included in the OTA upgrade package into the RADIO path of the temporary directory.
 - Example: To update the NON-HLOS.bin,
 - cp NON-HLOS.bin LA.UM.9.15.2/LINUX/android/out/dist/temp/RADIO/
- Generate the updated *merged-qssi_32go_bengal_32go-target_files.zip* with newly included images.
 - cd LA.UM.9.15.2/LINUX/android/out/dist/temp/
 - zip -qur LA.UM.9.15.2/LINUX/android/out/dist/merged-qssi_32go_bengal_32go-target_files.zip ./*
- Generate the full or incremental OTA upgrade package and push it into the target.

6.5 Build Script Changes to Include an Image as Part of the OTA

- 1 Steps to unzip, add HLOS/non-HLOS images, and zip the target file are already part of *build_system_ext.sh* script to copy the *recovery.img* as it's always needed for OTA.

```
#copy recovery.img to merged-qssi_32go_bengal_32go-target_files.zip
if [ -f $workdir/LA.UM.9.15.2/LINUX/android/out/dist/merged-qssi_32go_bengal_32go-target_files.zip ];then
  if [ -f $workdir/LA.UM.9.15.2/LINUX/android/out/target/product/bengal_32go/recovery.img ]; then
    if [ ! -d "$otadir" ]; then
      mkdir $otadir
    fi
    unzip -q $workdir/LA.UM.9.15.2/LINUX/android/out/dist/merged-qssi_32go_bengal_32go-target_files.zip -d $otadir/
    cp $workdir/LA.UM.9.15.2/LINUX/android/out/target/product/bengal_32go/recovery.img $otadir/RADIO/
    cd $otadir
    zip -qur $workdir/LA.UM.9.15.2/LINUX/android/out/dist/merged-qssi_32go_bengal_32go-target_files.zip ./
    cd $otadir/./
    rm -rf $otadir
  fi
fi
```

Figure 11:Building script

- 2 If any other image to be copied in addition to *recovery.img*, it can be added after the line below in *build_system_ext.sh*.

```
cp
$workdir/LA.UM.9.15.2/LINUX/android/out/target/product/bengal_32go/
recovery.img $otadir/RADIO/
```

- Make sure the *filesmap* file has an entry for this image to be copied.

7 Application Development

Use Android Studio to develop applications for the SE250B4 module.

For more information visit: <https://developer.android.com/studio>.

NDA Restricted

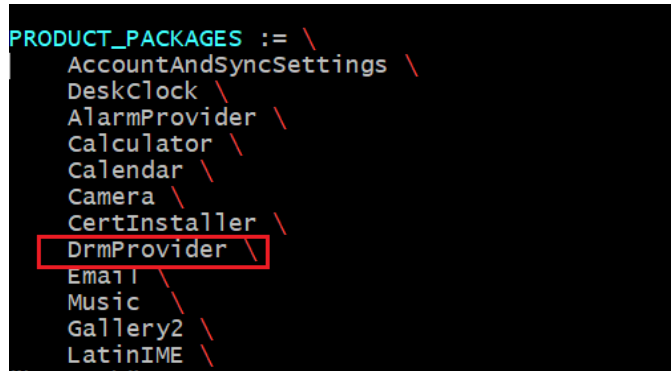
8 Android Customization

This chapter explains how to add or remove the built-in APKs in the SE250B4 Android system.

8.1 Case 1: Removing an Application by Modifying base.mk File

To remove the built-in APK "DrmProvider":

- 1 Open `/LA.QSSI.13.0/LINUX/android/device/qcom/qssi_32go/base.mk` and delete the entry for "DrmProvider" as shown below:



```
PRODUCT_PACKAGES := \
    AccountAndSyncSettings \
    DeskClock \
    AlarmProvider \
    Calculator \
    Calendar \
    Camera \
    CertInstaller \
    DrmProvider \
    Email \
    Music \
    Gallery2 \
    LatinIME
```

Figure 12: Delete the entry for "DrmProvider"

- 2 Rebuild the project using the following commands:

```
$cd ~/se250b4/build_scripts
```

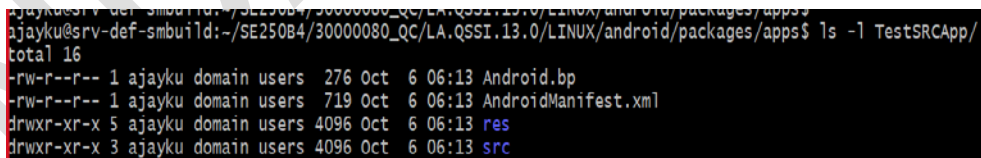
```
$ ./build_system_ext.sh --model=0UE0
```

After rebuilding, the "DrmProvider" APK will be removed from the build-in firmware.

8.2 Case 2: Adding an APK with Source Code into the System

To add an APK with source code (for example, **TestSRCApp.apk**) to the build-in APKs of the SE250B4 Android system:

- 1 Under `packages/apps`, create a new folder named after the APK to be added (for example, **TestSRCApp**).
- 2 Copy the source code of **TestSRCApp** APK to the Test folder.



```
ajayku@srv-def-smbuild:~/SE250B4/30000080_QC/LA.QSSI.13.0/LINUX/android/packages/apps$ ls -l TestSRCApp/
total 16
-rw-r--r-- 1 ajayku domain users 276 Oct 6 06:13 Android.bp
-rw-r--r-- 1 ajayku domain users 719 Oct 6 06:13 AndroidManifest.xml
drwxr-xr-x 5 ajayku domain users 4096 Oct 6 06:13 res
drwxr-xr-x 3 ajayku domain users 4096 Oct 6 06:13 src
```

Figure 13: Source code of TestSRCApp APK

- 3 In the **TestSRCApp** directory, create a file named **Android.bp** with the following contents:

```
android_app {
    name: "TestSrcApp",
    manifest: "AndroidManifest.xml",
    srcs: [
        "src/**/*.kt",
        "src/**/*.java",
    ]
}
```

```

],
sdk_version: "current",
static_libs: [
    "android-common",
    "androidx-constraintlayout_constraintlayout-solver",
    "androidx.appcompat_appcompat",
    "androidx.legacy_legacy-support-core-ui",
    "androidx.legacy_legacy-support-v4",
    "androidx.transition_transition",
    "androidx.recyclerview_recyclerview",
    "androidx.recyclerview_recyclerview-selection",
    "com.google.android.material_material",
    "androidx-constraintlayout_constraintlayout",
],
aaptflags: [
    "--auto-add-overlay",
    "--extra-packages androidx.constraint",
],
certificate: "platform",
}

```

- 4 Open /LA.QSSI.13.0/LINUX/android/device/qcom/qssi_32go /base.mk, and add Test to PRODUCT_PACKAGES:

```

PRODUCT_PACKAGES := \
    AccountAndSyncSettings \
    DeskClock \
    AlarmProvider \
    Calculator \
    Calendar \
    Camera \
    CellBroadcastReceiver \
    CertInstaller \
    DrmProvider \
    Email \
    Gallery2 \
    TestSRCApp

```

Figure 14: Install Steps

- 5 Rebuild the entire project, using the following commands:

```

$ ~/se250b4/build_scripts
$ ./build_system_ext.sh --model=0UE0"

```

Alternatively, use the "mm" command in the respective application folder to build only the application. Then traverse to the source directory and rebuild the whole project.

8.3 Case 3: Adding an APK Without Source Code into the System

To add an APK (for example, TestAPKs.apk) to the built-in APKs of the SE250B4 Android system, follow these steps:

- 1 Navigate to **packages/apps**, create a folder with the APK name (for example, TestAPK).
- 2 Place the **Test.apk** file inside the newly created **packages/apps/TestAPK** folder.
- 3 Create an **Android.mk** file in **packages/apps/Test** with the following contents:

```
LOCAL_PATH := $(call my-dir)
include $(CLEAR_VARS)

LOCAL_MODULE := TestAPK
LOCAL_SRC_FILES := $(LOCAL_MODULE).apk
LOCAL_MODULE_CLASS := APPS
LOCAL_MODULE_SUFFIX := $(COMMON_ANDROID_PACKAGE_SUFFIX)
LOCAL_CERTIFICATE := platform
include $(BUILD_PREBUILT)
```

Figure 15: Installation Steps

- Note:**
- If there is no **“.so”** file within apk, delete **local_ PREBUILT_JNI_ LIBS**.
 - If **“.so”** file exists, use **local_ PREBUILT_JNI_ LIBS** to list the paths to all **“.so”** files.
 - Use **@** to extract the **“.so”** files in APK and build them into the **Lib** folder under the same level directory of APK.

- 4 Open **/LA.QSSI.13.0/LINUX/android/device/qcom/qssi_32go /base.mk**, and add TestAPK to **PRODUCT_PACKAGES**:

```
PRODUCT_PACKAGES := \
    AccountAndSyncSettings \
    DeskClock \
    AlarmProvider \
    Calculator \
    Calendar \
    Camera \
    CellBroadcastReceiver \
    CertInstaller \
    DrmProvider \
    Email \
    Gallery2 \
    TestAPK
```

Figure 16: Product Packages

- 5 Rebuild the entire project, using the following commands:

Traverse to the source directory and rebuild the whole project using

```
$ cd ~/se250b4/build_scripts
$ ./build_system_ext.sh --model=0UE0
```

Note:

- If the app requires system-level permission, place it under `"/system/priv-app"` (default is under `"/system/app"`).
- If you add any permission to the apps, include them in the device file `"/system/etc/permissions/privapp-permissions-platform.xml"`.

```
LOCAL_PRIVILEGED_MODULE := true
```

- Modify `Android.mk`, to add `LOCAL_PRIVILEGED_Module := true` to ensure the app is placed under `/system/priv-ap`.

NDA Restricted

9 Acronyms and Abbreviations

Table 7: Acronyms and Abbreviations

Acronym	Definition
ADB	Android Debug Bridge
ADC	Analog – Digital Converter
API	Application Programming Interface
APK	Android Package Kit
CS	Chip Select
DDR	Double Data Rate
DSP	Digital Signal Processor
EDL	Emergency Download Mode
ELF	Executable and Linkable Format
GPT	GUID Partition Table
HLOS	High-Level Operating System
HSDPA	High-Speed Downlink Packet Access
HSUPA	High-Speed Uplink Packet Access
JNI	Java Native Interface
LPASS	Low Power Audio Sub-System
MSM	Mobile Station Modem
NA	North America
PCB	Printed Circuit Board
QFIL	Qualcomm Flash Image Loader
QPST	Qualcomm Product Support Tools
SDK	Software Development Kit
SIM	Subscriber Identification Module
SMP	Symmetric Multi-Processing
SPI	Serial Peripheral Interface
TE	Test Equipment
UART	Universal Asynchronous Receiver Transmitter
UMTS	Universal Mobile Telecommunication System
USB	Universal Serial Bus
WCDMA	Wideband Code Division Multiple Access
WLAN	Wireless Local Area Network

10 Related Documents

For current documentation and downloads, refer to <https://dz.telit.com/>.

Table 8: Related Documents

S.no	Doc Code	Document Title
1	1VW0301817	SE250B4 Smart Module Hardware Design Guide
2	1VW0301822	SE250B4 Smart EVB User Guide
3	1VW0301853	SE250B4 Quick Start Guide

NDA Restricted

11 Document History

Table 9: Document History

Revision	Date	Changes
4	2025-12-03	Template update
3	2025-07-29	Added Section 4.5.1 Steps to Prepare the Source code Section 4.6.1 Steps to Build the Software Updated section 4.5 Preparing Source Code and 4.3 Setting Up Build Tools
2	2025-05-23	Added Section 6. Building FOTA(Firmware Over The Air) Images
1	2025-02-14	Revised and standardised content throughout the document.
0	2024-06-24	Initial Release

From Mod.0809 Rev.11

NDA Restricted